

An AI-Driven, Multi-Criteria Decision Making Framework for Optimal Bug-to-Developer Assignment

System Architecture & Engineering Team

March 28, 2026

Abstract

The assignment of software defects to the most appropriate developer is a critical bottleneck in modern software engineering. Manual triage is time-consuming and prone to human error, often resulting in delayed patch times and misaligned resource allocation. This paper proposes a hybrid, automated bug-routing architecture combining Natural Language Processing (NLP), K-Nearest Neighbors (KNN), contextual Artificial Intelligence (AI) weighting, and the Weighted Sum Model (WSM). By mapping unstructured bug reports into a high-dimensional vector space, dynamically adjusting decision weights based on bug context, and evaluating developers through a multi-criteria utility matrix, this system mathematically guarantees the optimal developer assignment under strictly defined organizational constraints.

1 Introduction

In large-scale software systems, issue trackers receive hundreds of bug reports daily. The traditional triage process relies on human project managers to read, interpret, and assign these bugs based on their subjective understanding of the team's workload and expertise.

The proposed framework removes this bottleneck by introducing a deterministic, AI-assisted pipeline that evaluates developers across five dynamic axes: historical experience, average resolution speed, historical success rate, current workload, and domain-specific skill.

2 Methodology: The Assignment Pipeline

The algorithm operates in a sequential, eight-step pipeline designed to filter, evaluate, and rank candidates.

2.1 Bug Reporting & Feature Extraction

When a bug is reported, it consists of unstructured text (title, description) and structured metadata (severity, module). The system employs a pre-trained NLP model (such as

BERT or a TF-IDF vectorizer) to tokenize the text, remove stop words, and extract semantic features.

The bug is translated into a standardized feature vector, $\vec{V}_{bug}$, representing its position in a high-dimensional topic space.

2.2 Historical Similarity Search (KNN)

To find developers capable of solving the bug, the system queries a database of historically resolved bugs. It utilizes the K-Nearest Neighbors (KNN) algorithm, relying on Cosine Similarity to measure the angular distance between the new bug vector and historical bug vectors.

The similarity between the new bug $\vec{A}$ and a historical bug $\vec{B}$ is calculated as:

$$\text{sim}(A, B) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

The algorithm retrieves the top K most similar bugs. The developers who successfully resolved these K bugs form our initial candidate pool.

2.3 Candidate Identification and Hard Filtering

Not all historically successful developers are currently viable. A hard-filtering pass is applied to the candidate pool based on binary constraints:

- **Availability:** Is the developer currently on leave? (Boolean constraint)
- **Workload Threshold:** Does their current sprint capacity exceed 95%? (Boolean constraint)

Candidates failing these checks are immediately pruned from the decision matrix.

2.4 The Attribute Matrix Generation

For the remaining candidates, an attribute matrix is generated. Multi-Criteria Decision Making (MCDM) requires defining criteria as either “Benefit” (higher is better) or “Cost” (lower is better).

- **Experience (C_1):** Years of domain experience. (*Benefit*)
- **Fix Time (C_2):** Average hours taken to resolve similar bugs. (*Cost*)
- **Success Rate (C_3):** Percentage of code merged without causing regressions. (*Benefit*)
- **Workload (C_4):** Current utilization percentage. (*Cost*)
- **Domain Skill (C_5):** Extracted match rate between the bug’s NLP tags and the developer’s historical commit tags. (*Benefit*)

2.5 Dynamic Contextual AI Weighting

Static weights fail in dynamic environments. A minor UI bug requires a different assignment strategy than a critical database crash. A secondary, lightweight classification model assigns a weight vector $\vec{W}$ where $\sum w_j = 1$.

- **Critical Bugs:** Emphasize Fix Time (w_2) and Domain Skill (w_5).
- **Minor Bugs:** Emphasize low Workload (w_4) to distribute mundane tasks to junior developers, preserving senior bandwidth.

2.6 Normalization and WSM Calculation

Because attributes operate on different scales (e.g., Fix Time is in hours, Success Rate is a percentage), the data must be normalized using Min-Max scaling to a $[0, 1]$ range.

For **Benefit** criteria (Experience, Success, Skill):

$$x_{ij} = \frac{a_{ij} - a_j^{\min}}{a_j^{\max} - a_j^{\min}} \quad (2)$$

For **Cost** criteria (Fix Time, Workload):

$$x_{ij} = \frac{a_j^{\max} - a_{ij}}{a_j^{\max} - a_j^{\min}} \quad (3)$$

Where a_{ij} is the raw value of developer i for criterion j . Once normalized, the Weighted Sum Model (WSM) computes the final utility score U_i for each developer:

$$U_i = \sum_{j=1}^n w_j x_{ij} \quad (4)$$

2.7 Developer Ranking and Final Assignment

The system ranks all candidates descending by their final utility score U_i . The candidate with the maximum score represents the mathematical optimum for the given bug constraints. The system then automatically interfaces with the organization's ticketing software (e.g., Jira, GitHub) to assign the ticket, dispatching a notification payload to the selected developer.

3 Conclusion

By bridging semantic text analysis with deterministic operational research models (WSM), this algorithm entirely automates issue triage. The dynamic weighting ensures that the system mimics human intuition regarding urgency, while the strict mathematical models eliminate human bias, ultimately reducing the Mean Time to Resolution (MTTR) across software engineering organizations.